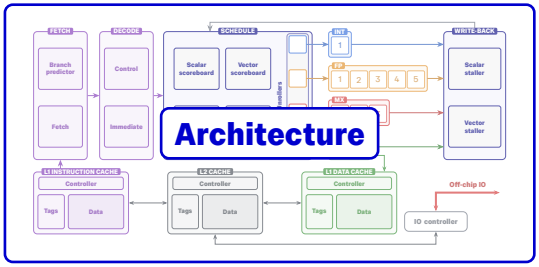
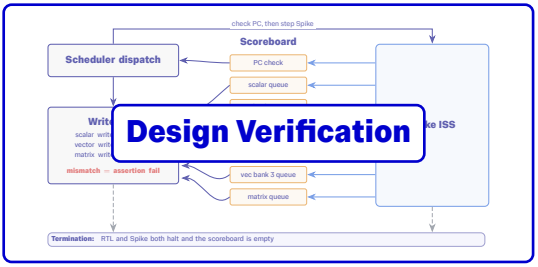
IMPERIAL

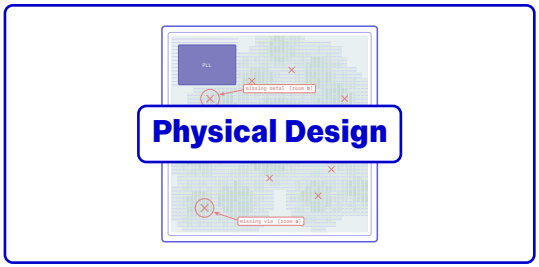
Auto-Regressive Inference Accelerator

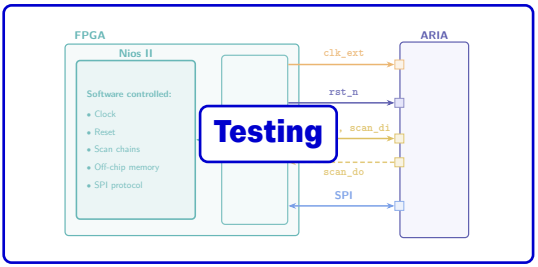
Digital VLSI Tapeout Project

Ilan Iwumbwe, Constantin Kronbichler, Petr Olšan, Ryan Voecks
18 June 2026


FETCH
DECODE
SCHEDULE
WRITE-BACK
Branch predictor
Fetch
Control
Immediate
Scalar scoreboard
Vector scoreboard
Scoreboarders
INT
1
MUL
1 2 3 4 5
MEM
Scalar staller
Vector staller
L1 INSTRUCTION CACHE
Controller
Tags
Data
L2 CACHE
Controller
Tags
Data
L1 DATA CACHE
Controller
Tags
Data
Off-chip IO
IO controller
Architecture

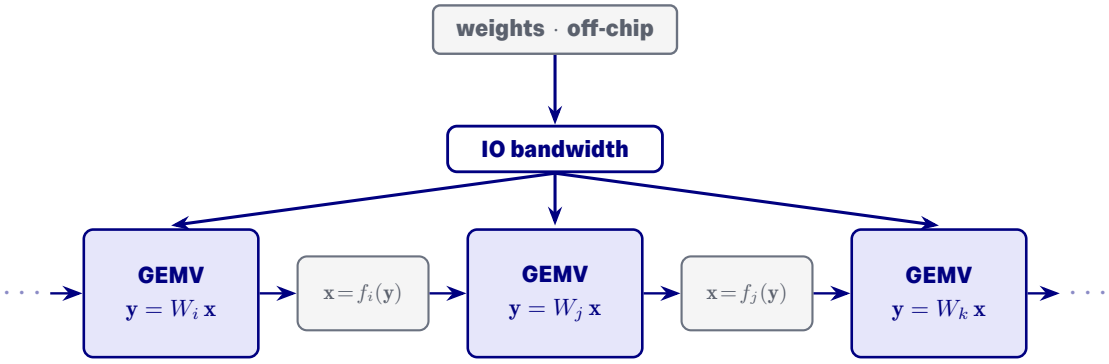

check PC, then step Spike
Scoreboard
Scheduler dispatch
PC check
scalar queue
vec bank 3 queue
matrix queue
mismatch → assertion fail
Termination: RTL and Spike both halt and the scoreboard is empty
Design Verification


PLL
missing metal (zone b)
missing via (zone a)
Physical Design


FPGA
Nios II
Software controlled:
• Clock
• Reset
• Scan chains
• Off-chip memory
• SPI protocol
ARIA
clk_ext
rst_n
scan_di
scan_do
SPI
Testing

Architecture

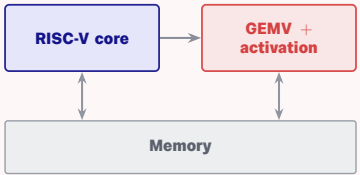
Single-sequence decode



Memory-bandwidth bound, not compute bound. Each weight read once, never reused.

CISC vs RISC

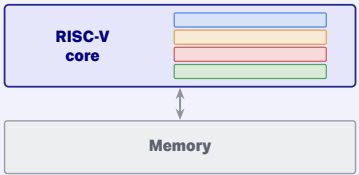
CISC · Dedicated GEMV unit



Accelerator driven by complex commands from simple RISC-V core

- Pairs with simple RISC-V core, all **complexity in accelerator**
- **Memory coherency** is challenging
- **Cannot be changed** after tapeout

RISC · Extended RISC-V core



Small custom instructions integrated into RISC-V core

- Programming and **optimisation** shifted to **after tapeout**
- **Shared** frontend and cache hierarchy
- RISC-V core becomes much **more complicated**

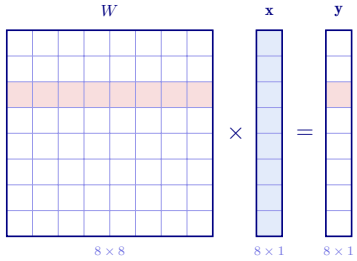
GEMV inner loop

```
# dot product
lw x, x_addr
lw w, w_addr
dot.fp8 xw, x, w

# calculate scale factor
lw scale_x_fp8, scale_x_addr
lw scale_w_fp8, scale_w_addr
cvt.fp32.fp8 scale_x, scale_x_fp8
cvt.fp32.fp8 scale_w, scale_w_fp8
fmul scale_xw, scale_x, scale_w

# accumulate
fmacc y, scale_xw, xw

# loop
addi i, i, 1
bne i, 256, loop
```

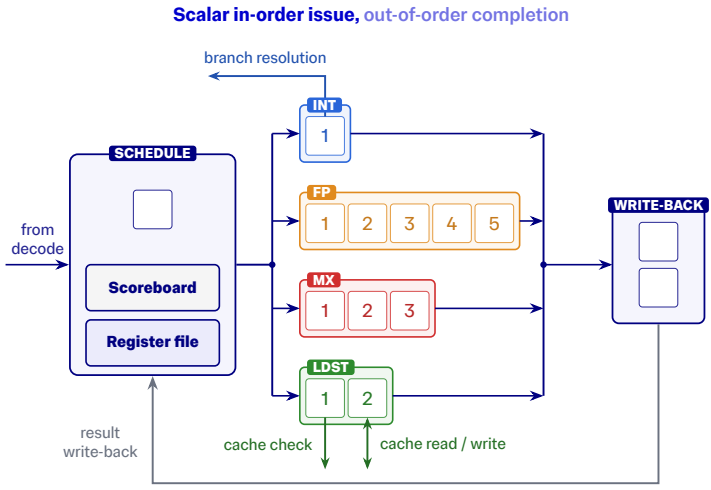


Mixed precision inference

- **Bandwidth bound:** 4× smaller means 4× faster
- **Scale factors cost extra multiplies:** trade memory for arithmetic intensity
- **Accumulation and activations in FP32;** downcast to FP8 for next GEMV

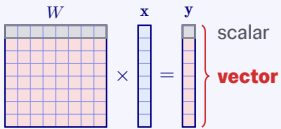
Scalar execution

```
1  # dot product
2  lw x, x_addr
3  lw w, w_addr
4  dot.fp8 xw, x, w
5
6  # calculate scale factor
7  lw scale_x_fp8, scale_x_addr
8  lw scale_w_fp8, scale_w_addr
9  cvt.fp32.fp8 scale_x, scale_x_fp8
10 cvt.fp32.fp8 scale_w, scale_w_fp8
11 fmul scale_xw, scale_x, scale_w
12
13 # accumulate
14 fmacc y, scale_xw, xw
15
16 # loop
17 addi i, i, 1
18 bne i, 256, loop
```



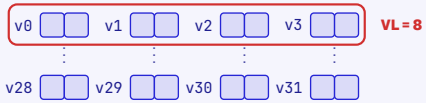
RISC-V vector architecture

Data parallelism



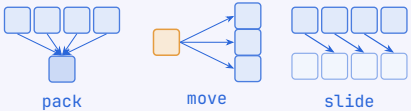
A **vectorised dot product** is a **GEMV**.

Vector registers and vector length



Vector registers contain **multiple elements**. Runtime configurable vector length (**VL**) controls **register grouping**.

Special instructions

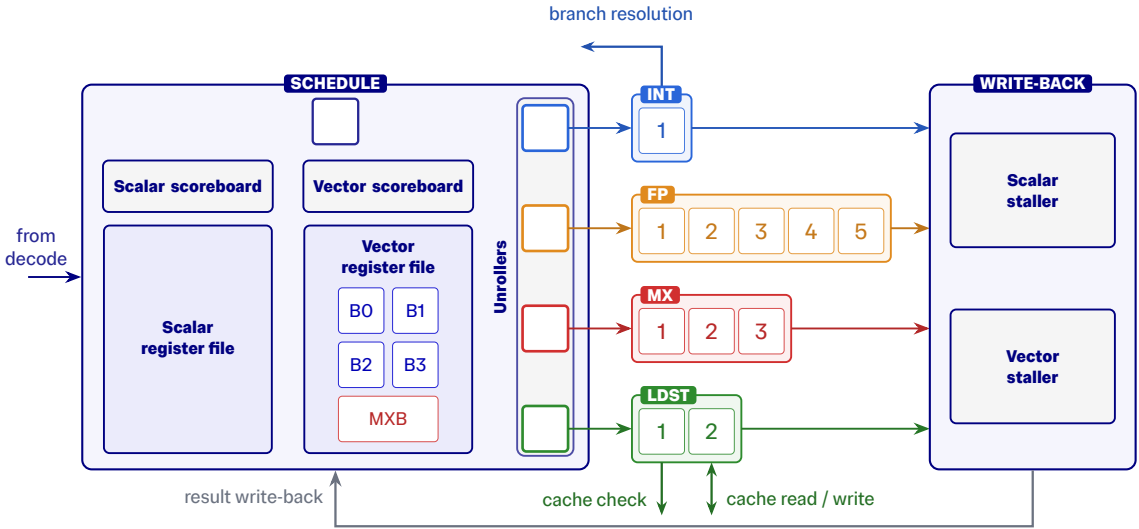


Custom data-movement ops: **pack** low-precision elements, **move** to/from scalar registers, **slide** between elements.

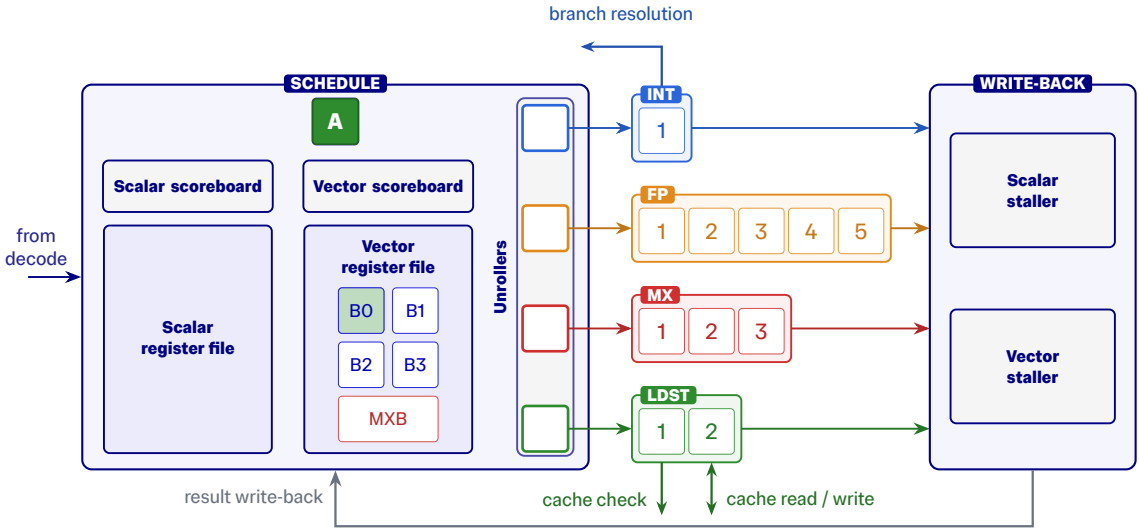
Multiplexing in space and time

Data parallel ops can be executed **in series** or **in parallel**.

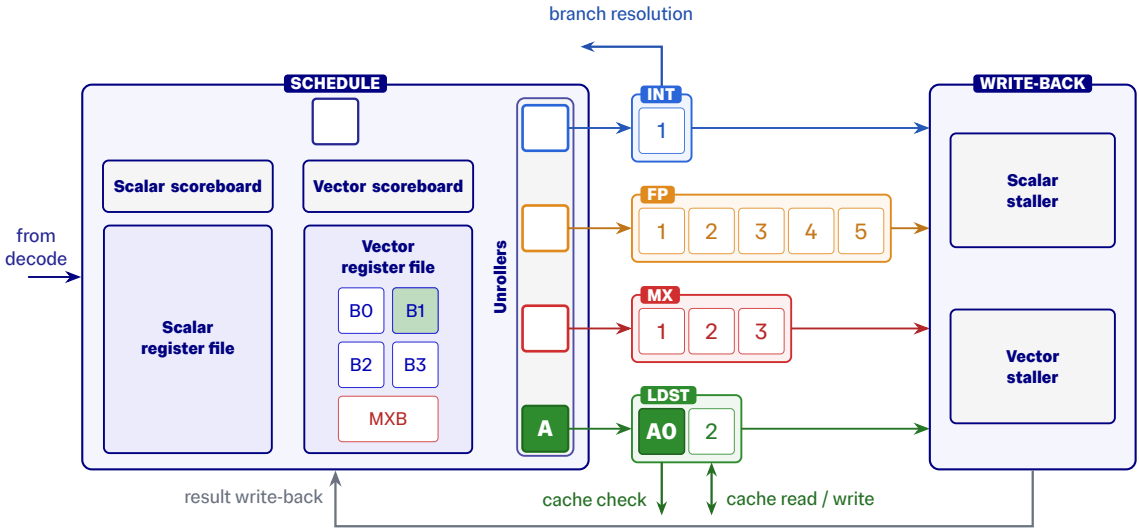
Vector execution



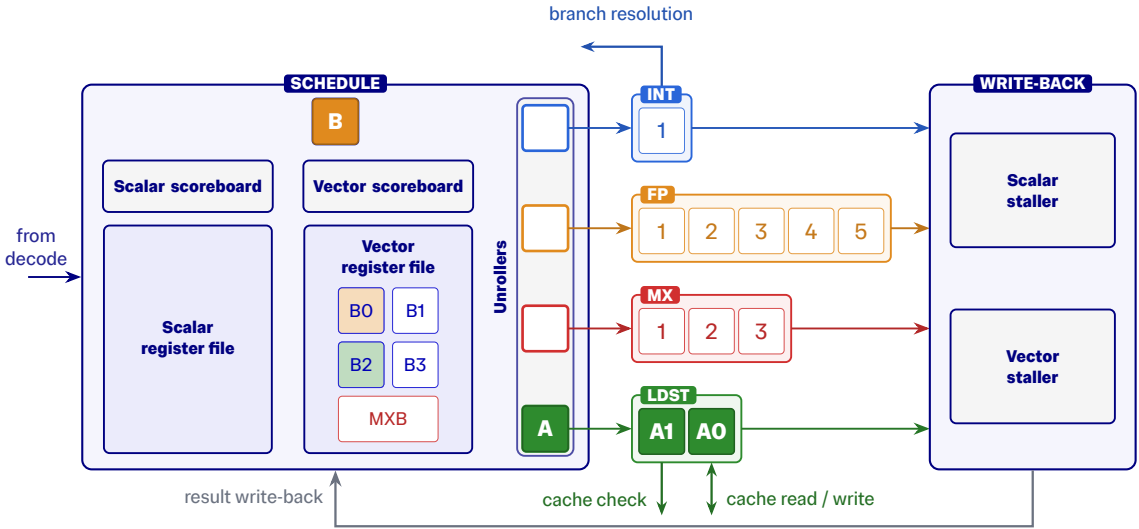
Vector execution



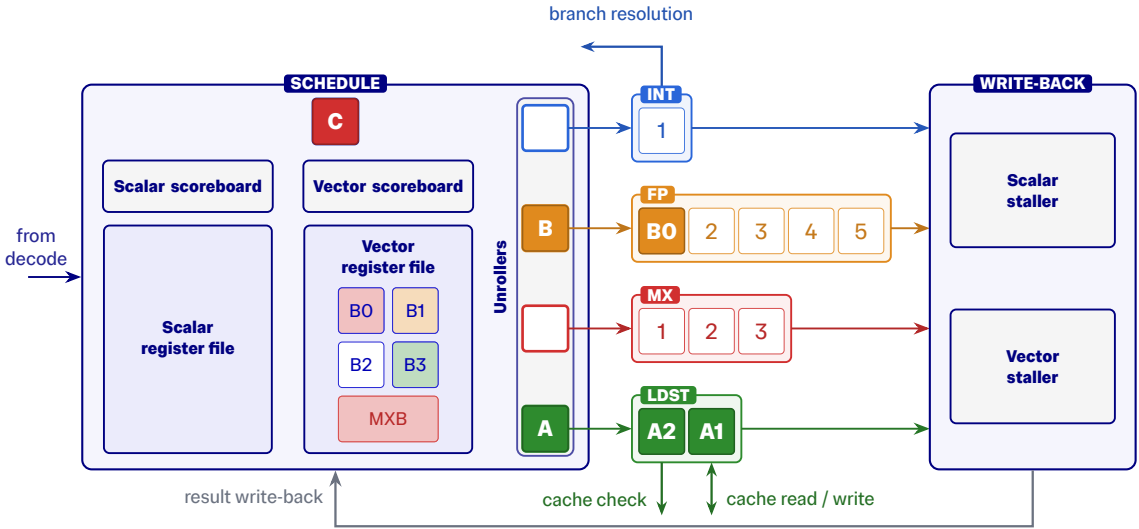
Vector execution



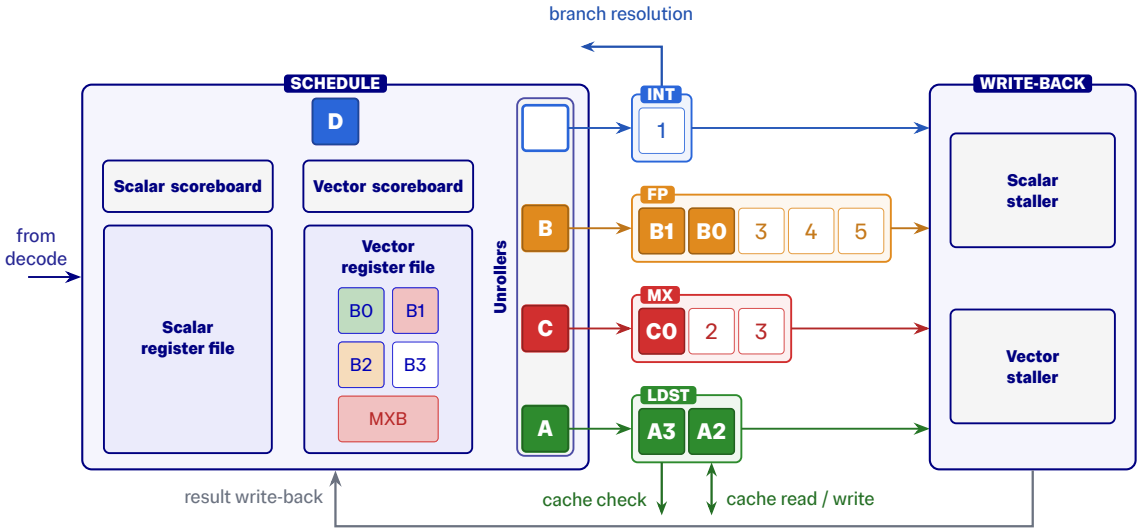
Vector execution



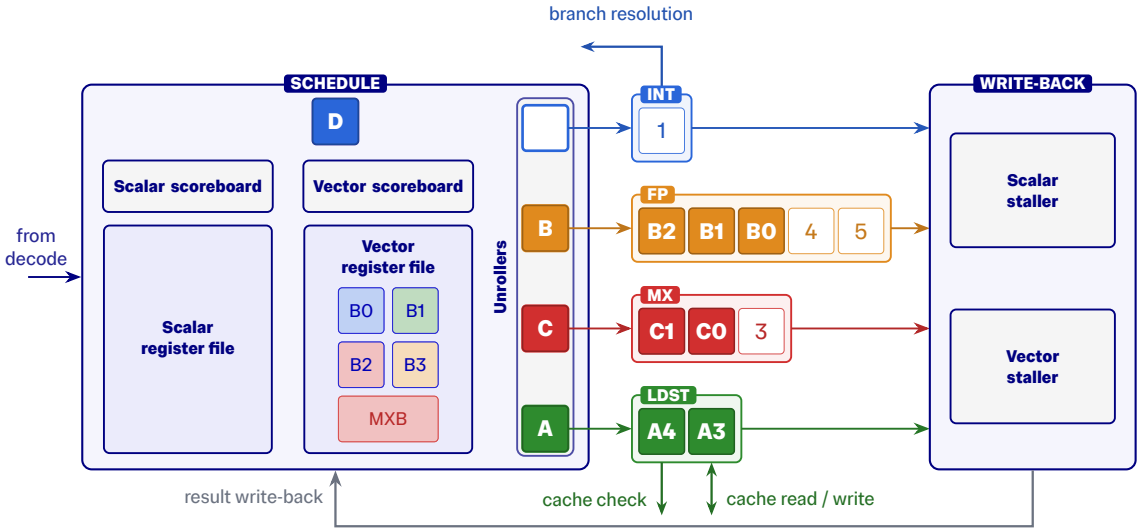
Vector execution



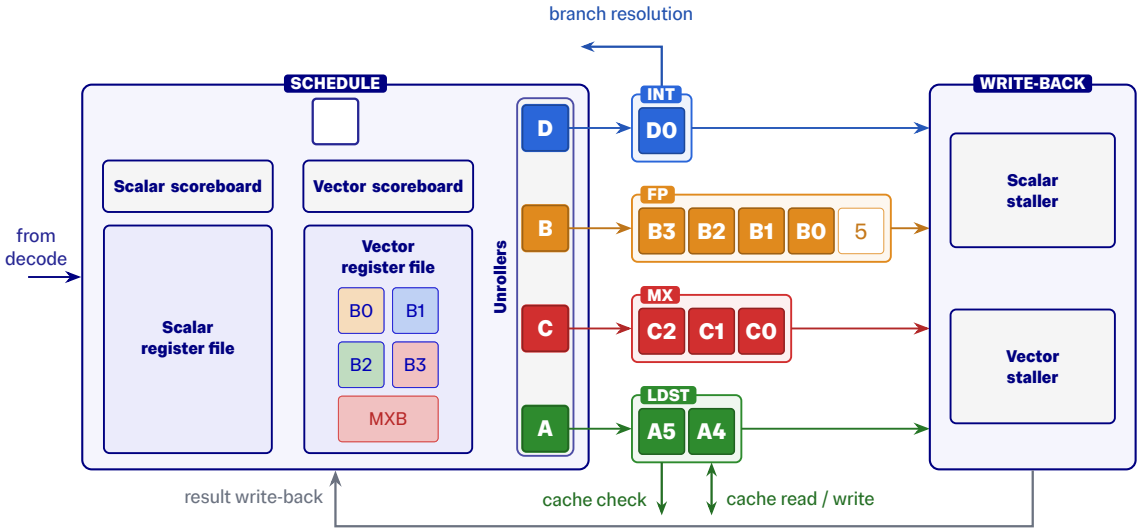
Vector execution



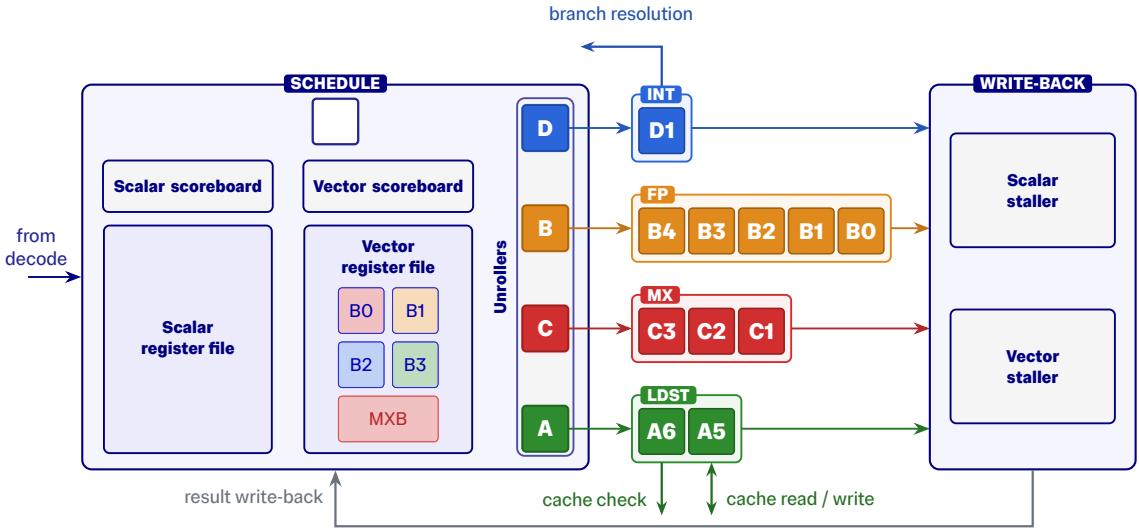
Vector execution



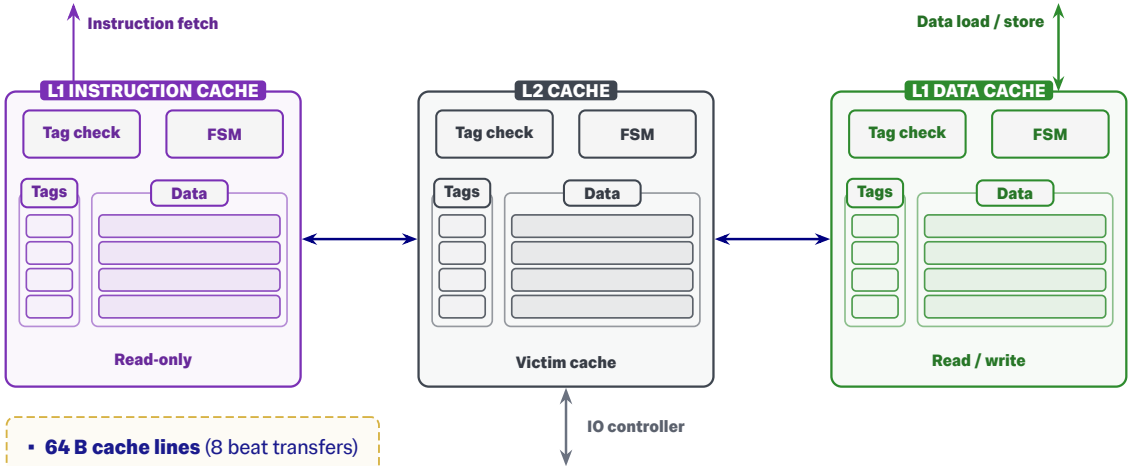
Vector execution



Vector execution

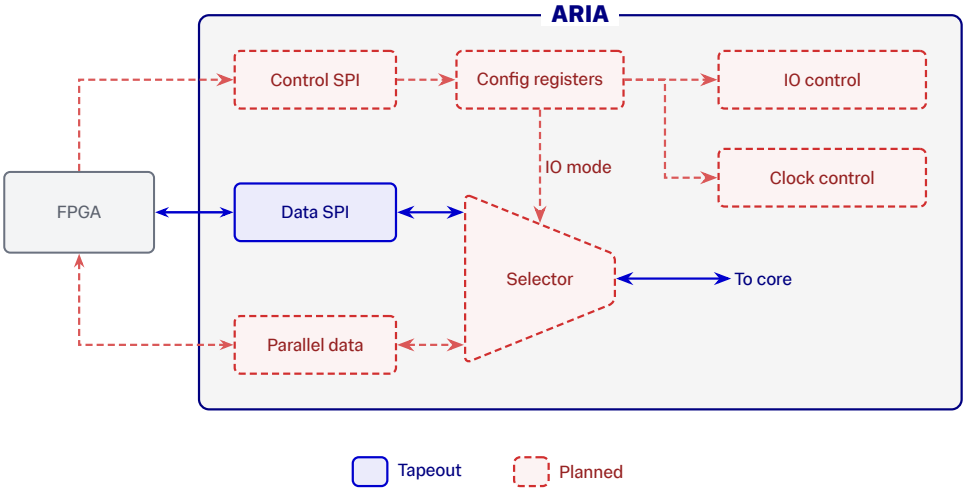


Cache hierarchy

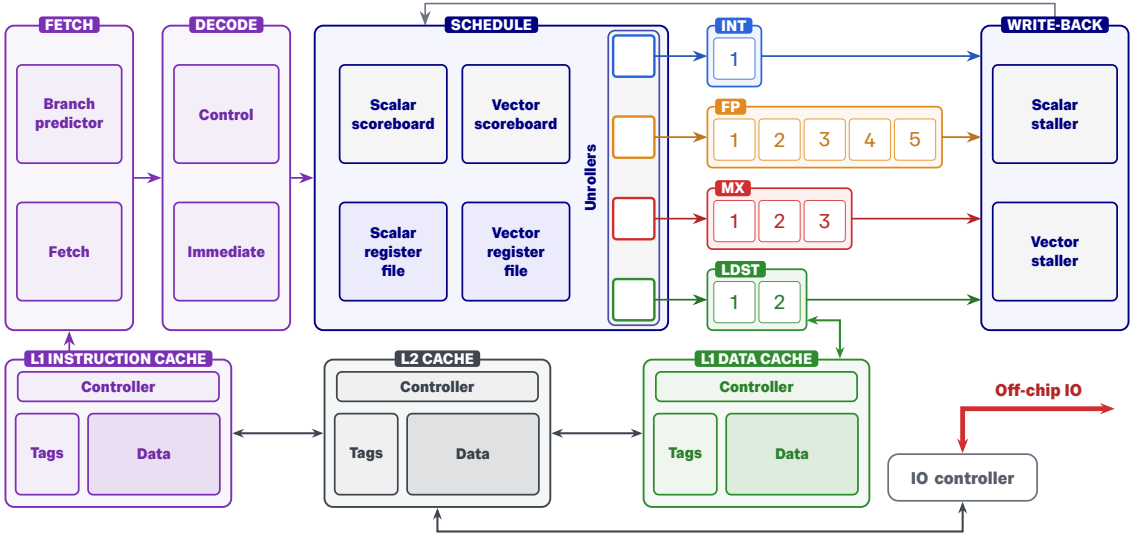


- **64 B cache lines** (8 beat transfers)
- **No prefetching**

IO controller and config



Architecture



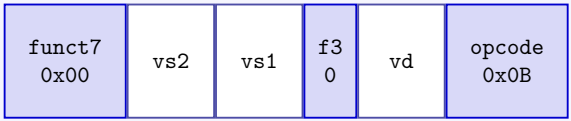
Design Verification

Integrating GCC

ISA definition

```
name: xa.vadd.vv
  ops: [vd, vs2, vs1]
  fields:
    opcode: 0x0B
    funct3: 0
    funct7: 0x00
```

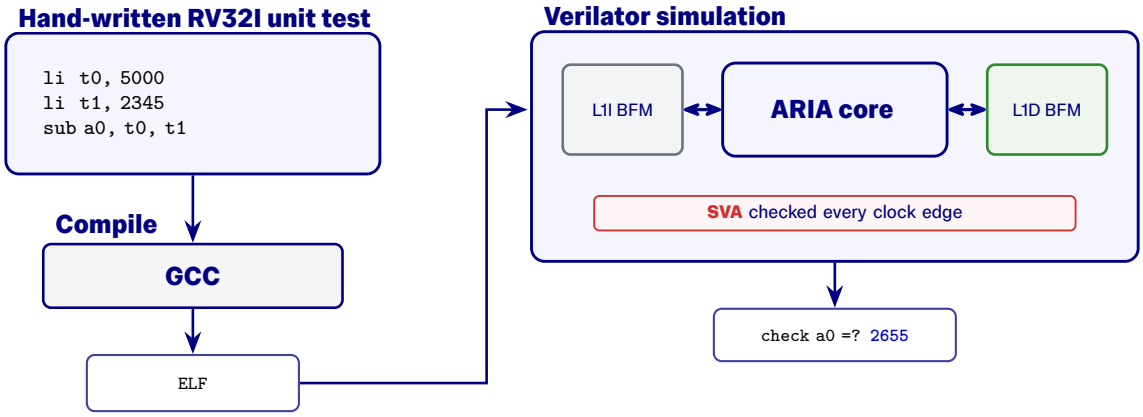
32-bit encoding



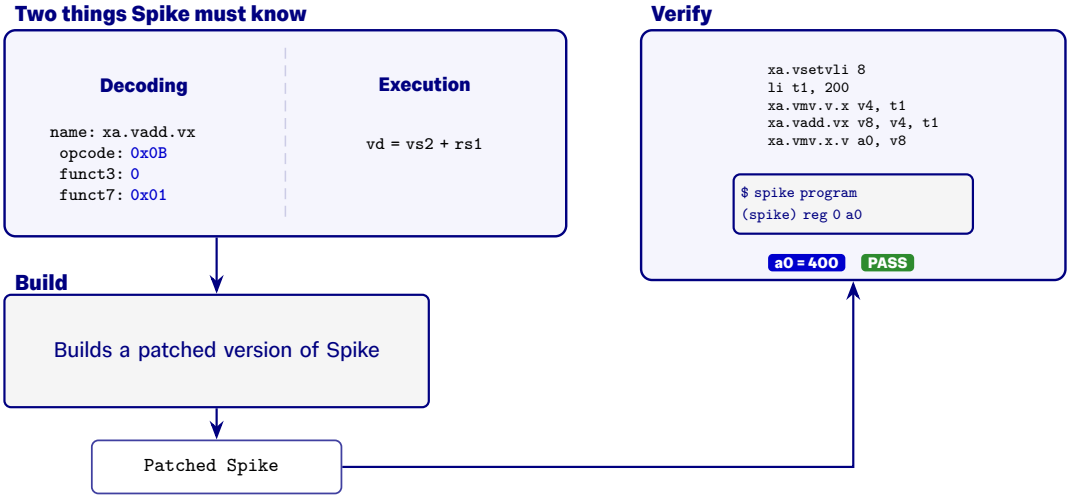
Code generation

Adds instruction identifiers and encodings to the compiler toolchain

Core unit tests

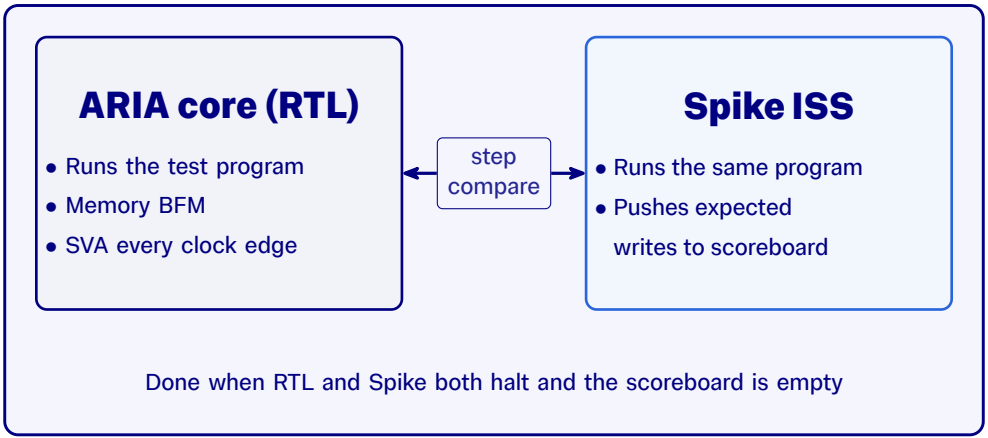


Integrating Spike

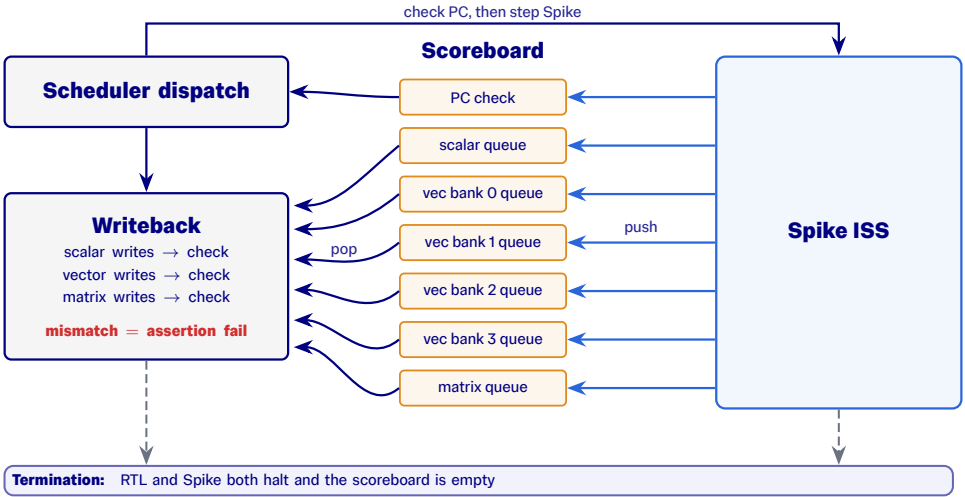


Spike co-simulation

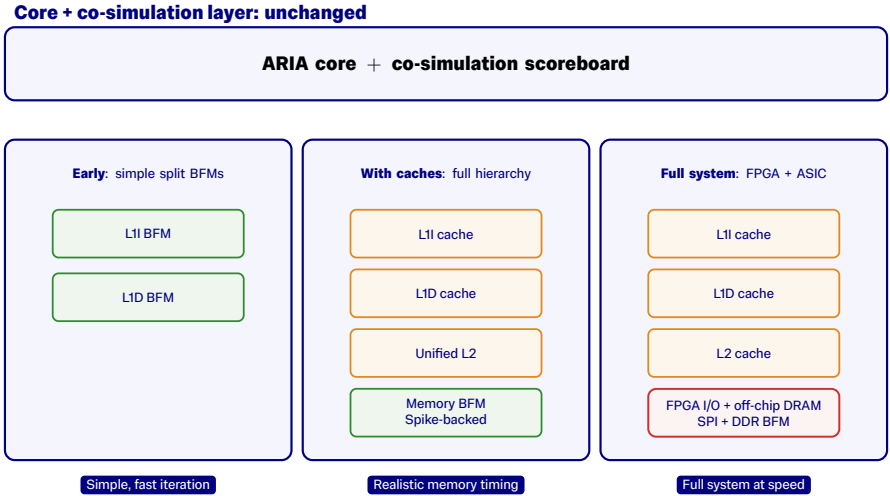
Co-simulation: RTL and Spike run the same program



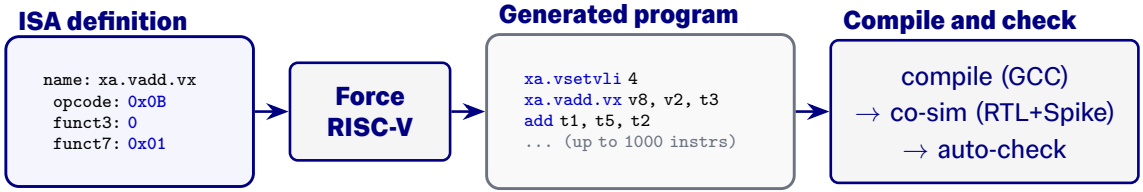
DPI-C bridge


check PC, then step Spike
Scoreboard
Scheduler dispatch
PC check
scalar queue
vec bank 0 queue
vec bank 1 queue
vec bank 2 queue
vec bank 3 queue
matrix queue
Writeback
scalar writes → check
vector writes → check
matrix writes → check
mismatch = assertion fail
pop
push
Spike ISS
Termination: RTL and Spike both halt and the scoreboard is empty

Scaling up


Core + co-simulation layer: unchanged
ARIA core + co-simulation scoreboard
Early: simple split BFMs
L1I BFM
L1D BFM
With caches: full hierarchy
L1I cache
L1D cache
Unified L2
Memory BFM
Spike-backed
Full system: FPGA + ASIC
L1I cache
L1D cache
L2 cache
FPGA I/O + off-chip DRAM
SPI + DDR BFM
Simple, fast iteration
Realistic memory timing
Full system at speed

Constrained random verification

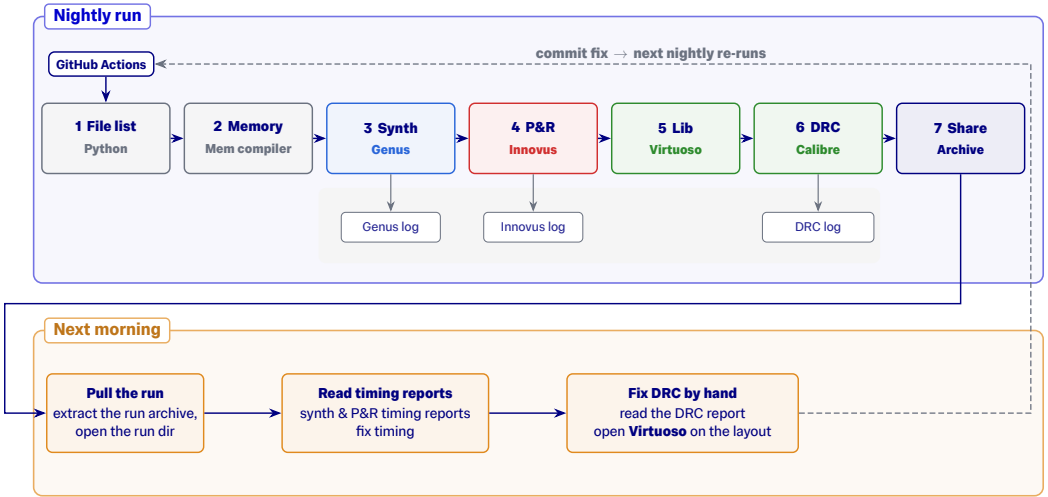


Coverage gaps

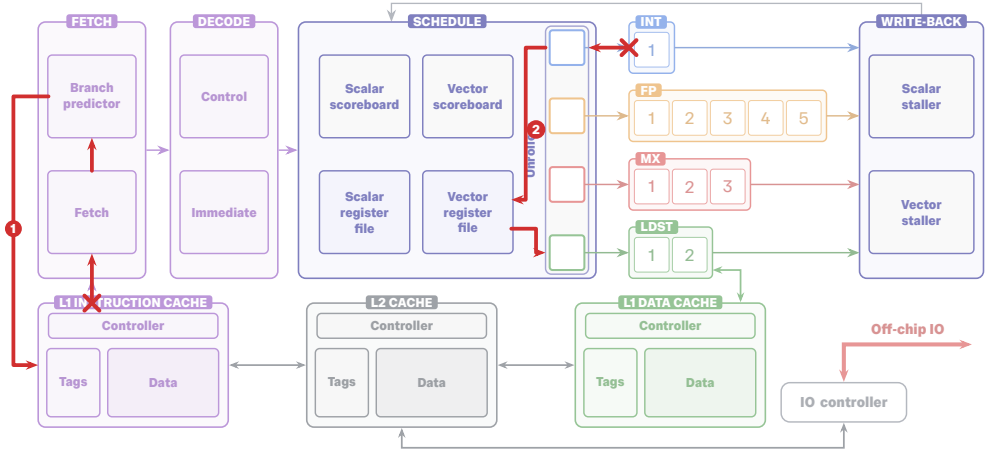
- **Vector alignment**: register index not even
- **Memory preamble**: load/store without a prior LUI/ADDI
- **Zero-base loads**: `rs1 = x0`
- **Small VL misalignment**: below-maximum vector load/store alignment
- **32-bit vector load**: needs 4-byte alignment

Physical Design

Automated PD flow



Critical paths

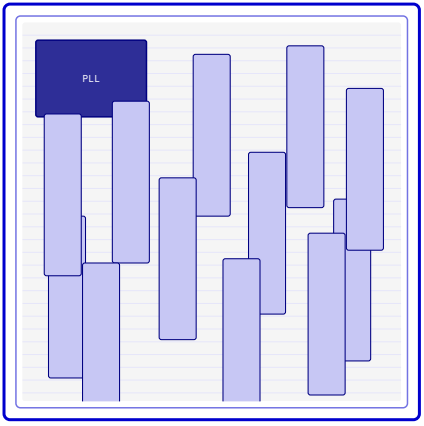


SRAM macro placement

(a) intended floorplan

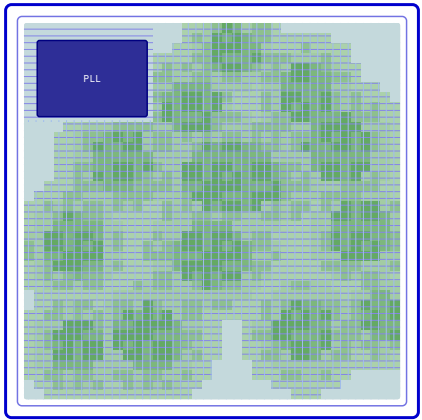
3 clusters × 4 SRAM macros each

(b) what the autoplacer produced



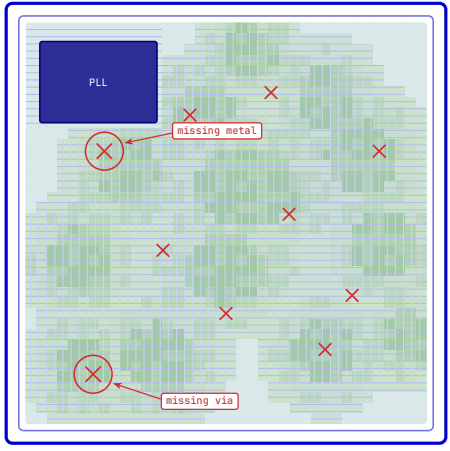
12 macros, scattered, no clustering

Final taped-out die



SRAM replaced with flops.
Dense logic and wiring fill core

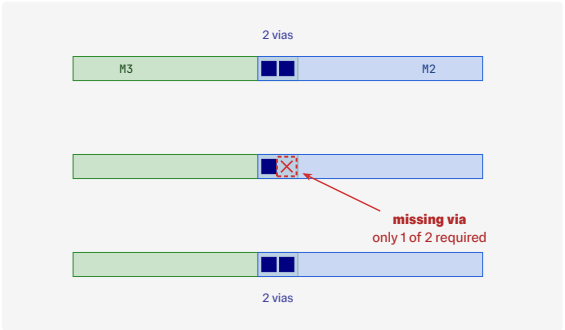
DRC violations across the die



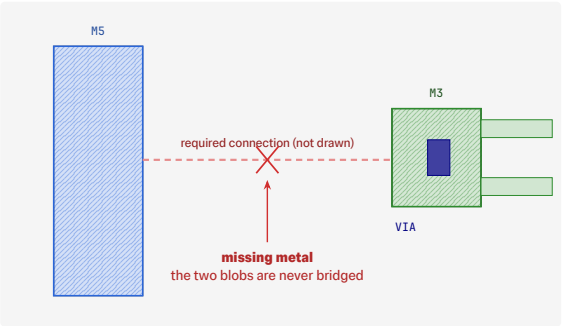
~**10 DRC violations** survived auto-fix

DRC violations up close

(a) missing via

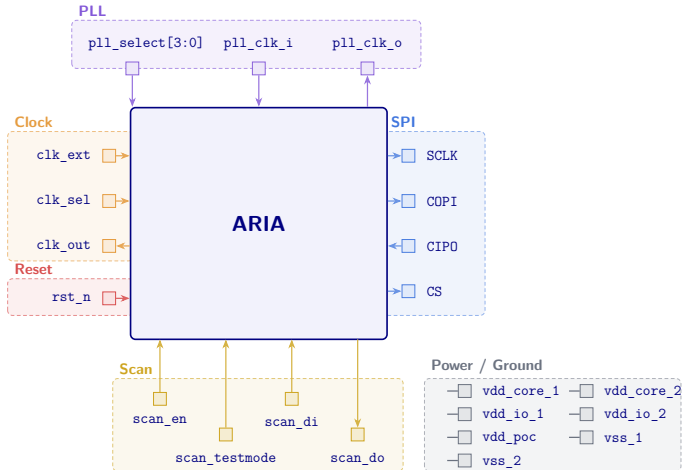


(b) missing metal



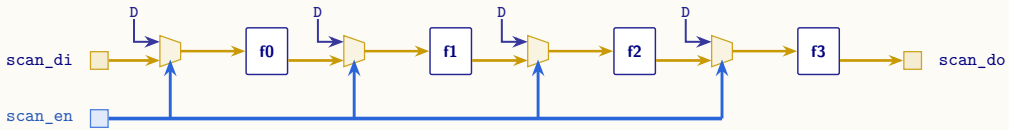
Testing

Test pins


PLL
pll_select[3:0]
pll_clk_i
pll_clk_o
Clock
clk_ext
clk_sel
clk_out
Reset
rst_n
ARIA
SPI
SCLK
COPI
CIPO
CS
Scan
scan_en
scan_testmode
scan_di
scan_do
Power / Ground
vdd_core_1
vdd_core_2
vdd_io_1
vdd_io_2
vdd_poc
vss_1
vss_2

Scan chains

Scan chain



Excluded from scan

- Scalar RF
- Vector RF (×4)
- L1 / L2
- Reset sync

Caveat

Excluded flops aren't gated while the chain shifts, so their contents corrupt.

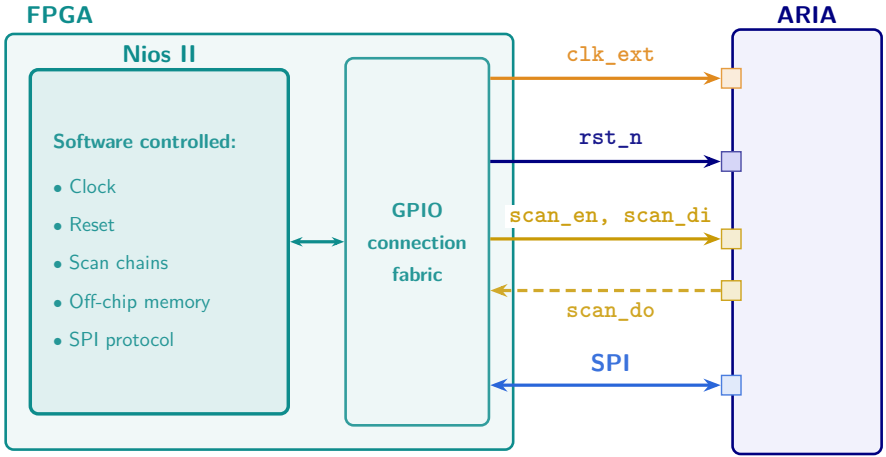
Fix: add an enable to disable them during a scan.

ATPG (Modus)

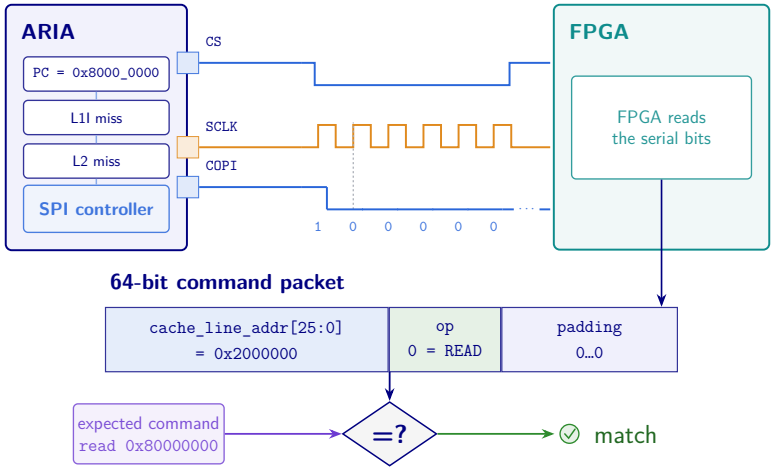
Fault-tests the gates, assuming logic between scanned flops is purely **combinational**.

Excluded flops inject `X`, but Modus copes if told which are off-chain.

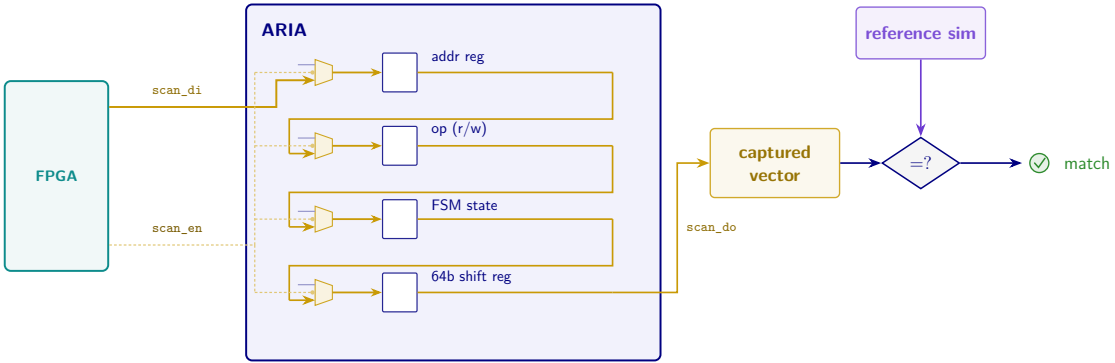
Test setup


FPGA
Nios II
Software controlled:
• Clock
• Reset
• Scan chains
• Off-chip memory
• SPI protocol
GPIO
connection
fabric
ARIA
clk_ext
rst_n
scan_en, scan_di
scan_do
SPI

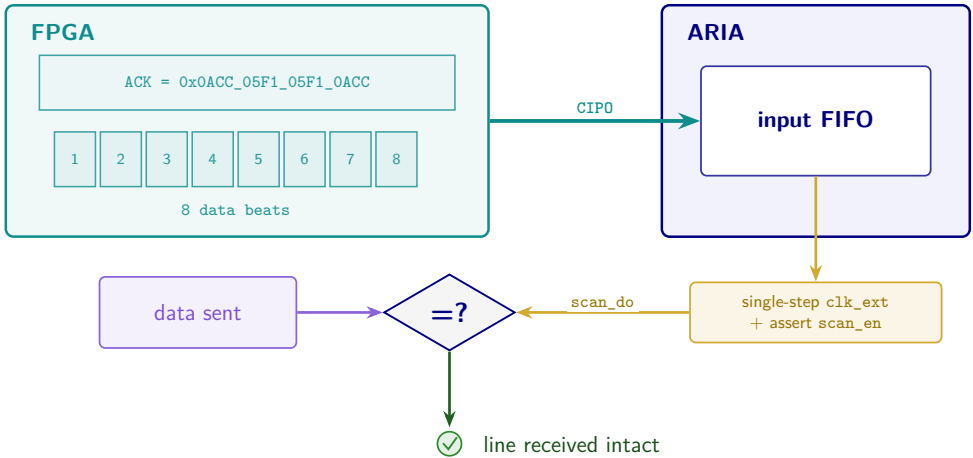
Phase 1: first request



Phase 2: internal state

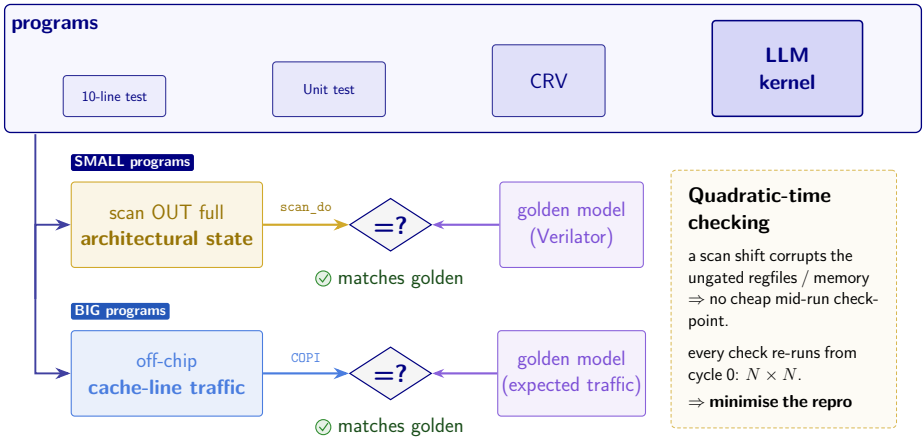


Phase 3: complete memory transaction



Phase 4: test programs

Run real programs, growing in size



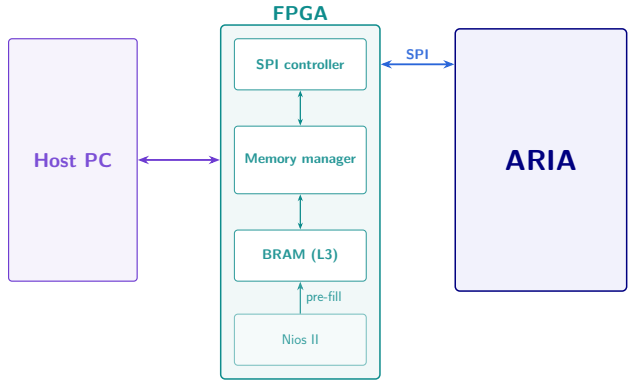
Quadratic-time checking

a scan shift corrupts the ungated regfiles / memory ⇒ no cheap mid-run checkpoint.

every check re-runs from cycle 0: $N \times N$.

⇒ **minimise the repro**

Phase 5: hardware control



From idea to silicon in three months

1 Architecture — vector RISC-V core, custom ISA extension, up to 4 uops/cycle

2 Verification — spike co-simulation, force-riscv constrained-random tests, SVAs

3 Physical design — automated nightly synthesis → P&R → DRC, final fixes by hand

4 Next: bring-up — FPGA-driven incremental testing with scan chains and SPI

IMPERIAL

Thank you.
Questions?